altairsim — Quick Reference

2026-09-04 · 59da746

Getting out, and back in

Key	Does
`^E`	**ATTN —** stop the machine and take the keyboard back. Nothing is lost.
`RUN`	Resume, at the exact instruction it stopped on.
`QUIT`	Leave. (There is no `EXIT`.)

Editing the command line

`Tab` completes what you are typing — a command, then a board id, then its property names, then a property's values (`SET mem0 fill=` then `Tab`); a second `Tab` lists the choices. `Up` / `Down` walk the command history, saved per directory in `.altairsim_history`.

Key	Does
`Ctrl-A` / `Ctrl-E` (or `Home` / `End`)	start / end of line
`Alt-B` / `Alt-F` (or `Ctrl-Left` / `Ctrl-Right`)	back / forward one word
`Ctrl-W` / `Ctrl-K` / `Ctrl-U`	erase word behind / to end of line / whole line
`Backspace` / `Delete`	erase before / under the cursor

Command line

```
altairsim [options] [machine]

  machine            a built-in name, or a config file (has a '/' or ends .toml).
                     Omitted: ./altairsim.toml if there is one, else `default`.
  -m, --machine <n>  ALWAYS a built-in name -- never a file.
  -f, --file <path>  ALWAYS a file -- never a built-in name.
  -n, --none         empty backplane: no boards, no memory, nothing.
  -l, --list         list the built-in machines and exit.
  -s, --script <f>   run a command script, then exit with its status.
  -x, --exec <cmd>   run one monitor command (repeatable), then exit.
  -i, --interactive  after --script/--exec, stay in the monitor.
      --mcp          MCP server on stdio.
  -v, --version      print the version and exit.
  -h, --help         print this help and exit.
```

Monitor commands

Type the part before the bracket.

Command	Does	Usage
BO[ARDS]	List, add, or remove boards on the backplane.	BOARDS [LIST]|ADD <type> <id> [k=v...]|REMOVE <id>
B[REAK]	Set a breakpoint on an address, memory/I/O access, or tape stop.	BREAK [<addr> | MEM R|W <addr> | IO R|W <port> | TAPE STOP] [IF <expr> | LOADS <expr>] [TRACE ON|OFF]
COM[PARE]	Compare a range of memory against another address.	COMPARE <range> <addr>
C[ONFIG]	Load or save the whole machine as a TOML file.	CONFIG LOAD <f.toml> | CONFIG SAVE <f.toml>
CONN[ECT]	Attach a serial unit to an endpoint (console, socket, file, ...).	CONNECT <id>:<u> <endpoint>
CONS[OLE]	Show or set the host console's properties.	CONSOLE [<k>=<v>...]
DE[POSIT]	Write bytes into memory at an address.	DEPOSIT <addr> <bytes...>
DI[SASM]	Disassemble memory into instructions.	DISASM [<addr>|<range>] [n] [CPU=8080]
DISC[ONNECT]	Unplug the endpoint from a serial unit.	DISCONNECT <id>:<u>
D[UMP]	Show memory as hex and ASCII.	DUMP [<addr>|<range>] [WIDTH=16]
E[DIT]	Enter bytes into memory interactively from an address.	EDIT <addr> [ROM]
EX[AMINE]	Point the front panel at an address (and show that byte).	EXAMINE [<addr>]
F[ILL]	Fill a range of memory with a byte.	FILL <range> <byte>
HE[LP]	Show help for a command.	HELP [<command>]
H[ISTORY]	Replay the recent instruction (or bus-cycle) history.	HISTORY [BUS|CPU] [n]
I[N]	Read a byte from an I/O port.	IN <port>
L[OAD]	Load a file into memory (binary, Intel hex or S-record).	LOAD <file> [AT <addr>] [FORMAT=BIN|HEX|SREC] [ROM]
M[OUNT]	Put a disk or tape image into a drive; a .imd is converted to a raw .dsk beside it.	MOUNT <id>[:<u>] <file> [WP] [CREATE] [extract[=<base>]] [k=v...]

`MOV[E]`	Copy a range of memory to another address.	`MOVE <range> <dest> [ROM]`
`N[EXT]`	Step one instruction, running any CALL/RST to completion.	`NEXT`
`NO[BREAK]`	Remove a breakpoint, or all of them.	`NOBREAK [id]`
`O[UT]`	Write a byte to an I/O port.	`OUT <port> <byte>`
`P[OWER]`	Power-cycle the machine -- the only thing that clears RAM.	`POWER`
`Q[UIT]`	Leave the simulator.	`QUIT`
`REGI[ON]`	Add a RAM or ROM region to a memory board.	`REGION ADD <id> type=ram|rom at=<addr> [size=|mount=]`
`RE[GS]`	Show the CPU registers (SET REG changes one).	`REGS | SET REG <r>=<v>`
`RES[ET]`	Reset the machine, keeping RAM (RESET CPU resets just the processor).	`RESET [CPU]`
`REST[ORE]`	Load machine state back from a snapshot.	`RESTORE <file>`
`R[UN]`	Start or resume the machine, optionally at an address.	`RUN [addr]`

`SA[VE]`	Write a range of memory out to a file.	`SAVE <file> <range> [FORMAT=BIN|HEX|OCTAL|PRN]`
`SEA[RCH]`	Find bytes or a string in a range of memory.	`SEARCH <range> <bytes...>|"str"`
`SE[T]`	Change a property of a board, the console, display, a register, or the bus.	`SET <id>[:<u>]|CONSOLE|DISPLAY|REG|BUS <k>=<v>`
`SH[OW]`	Display the state of a board, the bus, or the machine.	`SHOW <id>|BOARDS|BOARD <type> [UNITS]|MACHINES|MACHINE [<name>]|BUS [MAP|IO|IRQ|CONTENTION]|ROMS|MOUNTS|PATHS|CONSOLE|DISPLAY|SYMBOLS|VERSION`
`SN[APSHOT]`	Save the whole machine state to a file.	`SNAPSHOT <file>`
`S[TEP]`	Run one instruction (or n), showing the registers after each.	`STEP [n]`
`SY[MBOLS]`	Load or clear a symbol table for disassembly.	`SYMBOLS LOAD <file> [REPLACE] | SYMBOLS CLEAR`
`T[RACE]`	Log every bus cycle while the machine runs.	`TRACE ON|OFF [file] [MASK=IN,OUT,IRQ,DMA,CONTENTION]`
`TY[PE]`	Feed text to the guest as if typed at its keyboard.	`TYPE "text"`
`U[NMOUNT]`	Take a disk or tape out of a drive.	`UNMOUNT <id>:<u>`

`W[HO]`	Say which board answers an address or I/O port.	WHO <addr> | WHO IO <port>

Boards

CPU

Type	What it is
`6800`	Altair 680b CPU board: a Motorola 6800 at 500 KHz. Decodes nothing -- it drives the bus. The 88-CPU's twin, one core down, with memory-mapped I/O
`8080`	MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus
`8085`	Generic 8085 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with an 8085 core (RIM/SIM + TRAP/RST 5.5/6.5/7.5)
`z80`	Generic Z80 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core

Memory

Type	What it is
`bankmem`	S-100 bank-switched RAM. One card, four decoders (card=vector|cromemco64kz|northstar|expandoram2): a write-only select port swaps which RAM plane(s) drive the bus. Each card owns its own decode -- one-hot select (Vector 40), 8-bit bank mask (Cromemco 40), on/off+one-hot toggle (North Star C0), or PROM page-select (ExpandoRAM II FF, approximated)
`memory`	RAM/ROM board: a list of regions and PHANTOM* -- plain, unbanked memory (bank switching is its own board, `bankmem`)
`v2z80rom`	S100Computers V2 Z80 CPU board -- its onboard paged monitor EEPROM (the Z80 itself is board 'z80cpu'). An 8K 28C64 at F000-FFFF holding two 4K pages, builtin:master0 (low) / master1 (high), selected by OUT D3H bit1 (bit0=1 inactivates the EEPROM so RAM shows through). Shadows RAM in its window while enabled. Cold-start the MASTER monitor with startup=["RUN F000"]; the 'I' command boots CP/M 3 off a dualsd card

Disk

Type	What it is
`16fdc`	Cromemco 16FDC: WD FD1793 soft-sector floppy (single + double density), up to 4 drives. Disk registers at 30-34, a TMS 5501 console UART at 00-09 (unit 'tty'), and a 4K RDOS 2.52 boot PROM at C000 (OUT 40H banks it out, RESET restores it). Boots CDOS
`64fdc`	Cromemco 64FDC: the 16FDC's 1983 successor -- same FD1793 + TMS 5501, carrying an 8K RDOS 3.12 boot PROM at C000-DFFF (OUT 40H banks it out, RESET restores it). Boots CDOS
`dcdd`	MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits
`dualide`	S100Computers IDE-AB (CF): the IDE/CompactFlash half of the IDE+ESP32 combination board -- two CF sockets (drives 0/1 = A:/B:) for CP/M 3. Five 8255 ports at BASE+0..4 (default 30): A/B data, C control lines, mode config, drive select. Programmed-I/O ATA register engine (LBA read/write, 512-byte sectors). No boot PROM -- the CPU board's monitor boots CP/M from the CF. Mounts the SAME card image as dualsd (a .img with a .geo geometry sidecar); pair with dualsd for the full A:/B:+C:/D: system
`dualsd`	S100Computers Dual SD: two microSD sockets (drives 0/1) presented as raw 512-byte-sector CF/SD cards, for CP/M 3. Two ports at BASE+0..1 (default 80): status/command + data. Programmed-I/O command/handshake engine (33H-lead + 8 commands). No boot PROM -- the CPU board's monitor loads CP/M from track 0. Mount a card image (a .img with a .geo geometry sidecar)
`hdsk`	MITS 88-HDSK Datakeeper: Pertec hard disk, 256-byte sectors from a linear .DSK. Eight ports at BASE+0..7 (default A0). Command/handshake protocol, four page buffers
`icom`	iCOM FD3712/FD3812 8" floppy: a programmed-I/O command/handshake controller on the S-100 Interface board. Two ports at BASE+0..1 (default C0) plus a boot PROM and 6810 scratch RAM in high memory (rom=builtin:icom-fd3712-cpm | icom-fd3712-fdos | icom-fd3812-cpm). Boots CP/M 2.2 (single and double density) and FDOS. Up to 4 drives
`mds`	MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s
`tarbell`	Tarbell #1011: single-density WD FD1771 floppy, up to 4 drives. Eight ports at BASE+0..7 (default F8). Carries a 32-byte boot PROM that shadows 0000 over PHANTOM* -- boots CP/M automatically at reset (bootstrap=on)
`tarbelldd`	Tarbell #2022: double-density WD FD1791 floppy (mixed-density media, SD track 0), up to 4 drives. The single-density card's twin with a bitmap OUT-FC latch and a port-FD DMA/ext-addr register. Same 32-byte boot PROM
`versafloppy`	SD Systems VersaFloppy I/II: WD FD177x soft-sector floppy, up to 4 drives. Eight ports at BASE+0..7 (default 60). variant=vfi (FD1771, single density) | vfii (FD1791, single+double). Boots SDOS with the SBC-200 + DDBIOS

Serial

Type	What it is
`2sio`	MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3
`680io`	Altair 680b onboard I/O: a 6850 ACIA console ('tty') at F000/F001 and the config-strap read port at F002. Memory-mapped
`gsio`	Generic SIO: a strap-configurable serial board with TWO independent channels, units 'a' and 'b' (configure each under [board.unit.a] / [board.unit.b]). Basic transmit/receive only -- no programmable word length, parity, stop bits or framing/overrun status; a specific card that needs those is a separate emulated board. Per channel you strap: a status/control port (status_port -- read synthesizes DAV/TBMT at bit positions you pick, write is discarded) and a data port (data_port); one inverter_gate knob inverts both status bits together. Built-in profiles preset the straps: profile=sior0 (MITS SIO Rev 0, the default) | tuart (Cromemco TU-ART) | imsai-sio2 | compupro-if2 (CompuPro Interfacer II) | compupro-ss1 (CompuPro System Support 1). Channel a defaults to ports 0/1, b to 2/3. Polled, no interrupts. CONNECT each channel to a file, socket, serial port, in:/out:
`io4`	SSM IO-4 (2P+2S): the real Solid State Music board -- two full-duplex serial channels AND a four-port parallel section. Serial units 'a'/'b' are real 1602-family UARTs with programmable word length (data_bits 5-8), parity and stop bits (unlike the generic gsio), plus the full status-word strap-up (stat_* map, invert_status, port_reversal) and named profiles (default altair-rev1). A 4-port block set by switch S3 (default 0-3): Serial A status/data at BASE+0/+1, Serial B at BASE+2/+3. Parallel units 'pa'/'pb' are 8212 latched ports (input latch + service request, output latch) on their own 2-port block set by switch S4 (par_port, default 4-5): Parallel A at PAR+0, B at PAR+1; a byte the far end sends is strobed in, a write latches out, and the §3.2.2 status/data console flag is strappable (dav_bit/dav_source/dav_active_low). If the two blocks overlap, neither section answers the shared ports. Each serial channel's receive (DAV) and transmit (TBMT) and each parallel input (service request) can raise an interrupt, strapped on header W4 to a VI line, pin 73 (int), or none (rx_int/tx_int per serial channel, int per parallel port) -- there is no software enable, the strap is the enable, and a parallel input interrupt rises even when the port is not addressed. Configure each unit under [board.unit.a] / [board.unit.pa] etc.; CONNECT each to a file, socket, serial port, in:/out:
`pmmi`	PMMI MM-103: Bell 103 modem on an S-100 card, unit 'line'. Four ports at BASE+0..3 (default C0), read/write different registers. Transmit/receive over a ByteStream; CONNECT it to in:/out: files. No dialer; modem status is a fixed stub
`propio`	S100Computers Console IO Board (Parallax-Propeller console), unit 'serial'. A strap-configurable serial subtype preset to the board's documented convention: status/data at 00/01, RX-ready = status bit 1, TX-ready = status bit 2, both active high. Every strap (status_port/data_port/dav/tbmt/inverter_gate) is still overridable -- the real board is jumpered. Polled, no interrupts. CONNECT it to a file, socket, serial port, in:/out:
`sbc`	SD Systems SBC-100/200: Z80 single-board computer. One 8-port block (78-7F): Intel 8251 console (unit 'tty', data 7C / status 7D, RxD->/DSR auto-baud for MSMONR21), Z80-CTC (78-7B) whose ch1 raises a mode-2 keyboard interrupt (vector 0x82) off the 8251 RxRDY, and a parallel port (7E/7F) whose OUT 7F bit 1 switches the onboard PROM out. Optional onboard boot PROM via [[board.socket]] (at+mount). variant=sbc100|sbc200
`sio`	MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits

`turnkey`	MITS 8800b Turnkey Module: phantom boot PROM (FC00-FFFF), integrated 6850 SIO (unit 'tty', default 0x10), sense switches at FF, and the Auto-Start JMP jam. Sockets via [[board.socket]]

Tape

Type	What it is
`680kcacr`	Altair 680b KCACR audio-cassette interface: a 1602-family UART recording Kansas City Standard FSK, memory-mapped at F010 (status/control) and F011 (data), active-LOW. Adds software motor control (control D7=on, D6=off) and interrupt-driven transfer (D0/D1 enables pull the 6800 IRQ). Reuses the 88-ACR tape machinery -- MOUNT a tape, WIND/REWIND it
`acr`	MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the WIND/REWIND/EXTRACT verbs and a tape counter
`uio`	MITS 88-UIO: serial + cassette on one board. A 6850 (unit 'serial', default 0x10) and an 88-ACR cassette section (unit 'tape', default 0x06) with motor control and a SW-1 MITS/Kansas-City modulation switch. Defaults reproduce the standard 0x10 + 0x06 layout

Parallel and printer

Type	What it is
`4pio`	MITS 88-4PIO: up to four 6820 PIAs, sections ja/jb.. per port. 16 ports from BASE (default 20). Software-set direction; CONNECT each section
`680uio`	Altair 680b Universal I/O: a second 6850 ACIA serial port ('serial') and a 6820 PIA parallel port (sections 'p1a/p1b', 'p2a/p2b' with pias=2) in an S9-relocatable window (default base F000: serial F006/F007, PIA F008-F00F), plus fixed switch inputs at F003 and a non-latched output at F010-F013. Memory-mapped, active-high
`c700`	MITS 88-C700: Centronics line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Output-only; CONNECT it to a file, a socket, or a real printer queue
`d7a`	Cromemco D+7A: analog + parallel I/O. Eight ports from BASE (default 18): one parallel port + seven two's-complement A/D-in/D/A-out channels. Reads 1-2 JS-1 joysticks from the host
`lpc`	MITS 88-LPC: 88-LP line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Line-buffered: 6-bit codes + PRINT/LINE FEED/CLEAR. CONNECT it to a file, a socket, or a real printer queue
`pio`	MITS 88-PIO: 8-bit parallel port, units 'out'/'in'. Two ports at BASE+0..1 (default 04). CONNECT a printer, a keyboard, a socket

Video

Type	What it is
`dazzler`	Cromemco Dazzler: color graphics from a framebuffer in main RAM. Two ports at BASE+0..1 (default 0E): control/status and format. 32x32 to 128x128, 16 colors/greys. Needs a Display
`vdb8024`	SD Systems VDB-8024: an 80x24 video terminal on one board -- the video console for an SBC-100/200 (the alternative to the 8251). Two I/O ports at BASE+0..1 (default 00): status/keyboard/display. Unit 'keyboard' (CONNECT). Optional keyboard-strobe interrupt strap (interrupt=vi0..vi7) for the SBC-200's CTC to vector -- what the SD video CBIOS needs; polled by default. Boots sdmonv21. Needs a Display
`vdm1`	Processor Technology VDM-1: memory-mapped 16x64 video, screen RAM at BASE (default CC00), scroll/status port (default CC). Needs a Display

Systems

Type	What it is
`sol`	Processor Technology Sol-PC I/O: serial, keyboard, parallel, CUTS tape as one board. Seven ports F8..FE. Units serial/printer/keyboard (CONNECT) and tape1/tape2 (MOUNT). Brings the WIND/REWIND/EXTRACT verbs and a tape counter

PROM programmer

Type	What it is
`pb1`	SSM PB1: 2708/2716 EPROM programmer + on-board EPROM board. A 4K programming-socket window (default D000, sockets U22=2708/U23=2716) and one control port (default 10): OUT arms the board and picks the chip (D0=2708, D1=2716), then a window write burns a byte and a window read disarms it. Save the burn to a host hex file with `SAVE file window`. Optional read-only on-board EPROM area via [[board.prom]] (at + mount). The board ships no firmware -- run any 2708/2716 burner; SSM's own from the PB1 manual is in examples/pb1

Other

Type	What it is
`fp`	Altair front panel: the SENSE switches a guest reads at IN 0FFH -- a configured byte (SET fp0 sense= or TOML), not toggled here. No OUT
`hostbridge`	Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN BOARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM
`ss1`	CompuPro System Support 1: multifunction S-100 board. Dual 8259A interrupt controllers in a master/slave cascade (master/slave at base+0..+3; master watches VI0-6 and drives pin 73, slave takes the timer OUTs and the UART's Rx/TxRDY), an 8253 interval timer (three counters + control at base+4..+7, 2 MHz clock), the OKI MSM5832 battery-backed real-time clock/calendar (command/data at base+10/+11) and a 2651 UART serial channel (base+12..+15); base default 50H. The 9511/9512 math socket is unpopulated
`virtc`	MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE

Machines

Machine	What it is
`8085`	A minimal 8085 machine: an `8085` CPU, 64K of RAM, and a 2SIO console.
`acuter`	ACUTER at F000 -- CUTER on a plain Altair, with a terminal instead of a VDM.
`altair680`	The Altair 680b -- MITS's second machine, and a different animal from the 8800.
`altmon`	An Altair with a monitor in ROM and a terminal on it.
`amon`	AMON 3.1 in a 4K EPROM at F000 -- Martin Eberhard's full-featured Altair monitor.
`bankmem`	A bank-switched RAM machine: a Z80, a console, and a Vector Graphic 64K bankmem.
`basic4k`	The machine Altair 4K BASIC was sold to run on: an 88-SIO Teletype, a cassette in the ACR.
`basic8k`	The machine Altair 8K BASIC was sold to run on: an 88-2SIO terminal, a cassette in the ACR.
`cdbl`	The `default` machine with the Combo Disk Boot Loader in the PROM socket.
`compupro`	A stock Altair with a CompuPro System Support 1 board for its clock/calendar.
`cuter`	CUTER 1.3 driving a Processor Technology VDM-1 -- the real Sol/CUTS monitor.
`dazzler`	A Cromemco Dazzler in an Altair -- the bench for the S-100's first color graphics card.
`default`	The machine you get when you name none: 56K, and the DBL boot PROM at FF00.
`dualide`	S100Computers "IDE-AB CF" machine -- a V2 Z80 CPU board booting CP/M 3 off a CompactFlash card.
`dualidesd`	S100Computers "IDE-AB CF+ESP32" machine -- both boards, CP/M 3 on CF drives A:/B: and SD drives C:/D:.
`dualsd`	S100Computers "Dual SD" machine -- a V2 Z80 CPU board booting CP/M 3 off a microSD card.
`icom`	iCOM FD3712 8" floppy machine -- boots CP/M 2.2 off a single-density iCOM disk.
`lineprinter-lpc`	The `default` machine with an 88-LPC line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`lineprinter`	The `default` machine with an 88-C700 line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`minidisk`	The Altair Minidisk: an 88-MDS at 08 and the MDBL boot PROM. You supply the 5.25" disk.
`original`	The Altair as it actually left Albuquerque.
`parallel`	The `default` machine with two MITS parallel boards: an 88-PIO and an 88-4PIO.
`ps2`	The machine MITS Programming System II ran on: basic8k's cards, but not basic8k's bootstrap.
`ps2int`	MITS Programming System II, WITH INTERRUPTS. `ps2` with A9 down and an 88-VI/RTC in it.
`rombasic`	MITS Extended ROM BASIC 16K -- Extended BASIC that runs directly out of ROM.

`sbc200`	SD Systems SBC-200 -- a 4 MHz Z80 single-board computer running the MSMONR21 monitor.
`sbc200v`	The SD Systems SBC-200 with a VDB-8024 VIDEO console: SDMONV21 on an 80x24 screen.
`sol20`	The Processor Technology Sol-20 -- an integrated 8080 machine, running SOLOS.
`tarbell`	Tarbell #1011 single-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`tarbelldd`	Tarbell #2022 double-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`turnkey`	The MITS 8800bt -- an Altair with a Turnkey Module where the front panel used to be.
`vdm1`	A Processor Technology VDM-1 in an Altair, and a demo that draws on it.
`z80`	A minimal Z80 machine: a `z80` CPU, 64K of RAM, and a 2SIO console.

A machine file, in one look

```
[machine]
name    = "mine"
base    = "default"        # start from a machine, and say what is DIFFERENT
startup = ["RUN FF00"]     # the operator's own keystrokes. There is no BOOT verb.

[[board]]                  # type + a NEW id      -> ADD the card
type = "2sio"              # type + an id from the base -> REPLACE it outright
id   = "sio0"              # NO type + an id      -> MODIFY the one already there
port = 10                  # remove = true        -> PULL THE CARD OUT

  [board.unit.a]           # a unit's own settings
  connect = "console"

  [[board.region]]         # a list the card owns (memory)
  type = "ram"
  at   = 0000              # hex: it is an address
  size = "56K"             # decimal: it is a size

  [[board.drive]]          # a list the card owns (disk controllers)
  unit  = 0
  mount = "cpm.dsk"        # relative to THIS FILE

[console]                  # the HOST's terminal -- not a board
strip7out = true
base      = octal          # read/print the wire class in split octal (MITS style)
```

Paths: a path *inside* a machine file is relative to **that file**. A path you *type* is relative to **your shell**.

Endpoints — `CONNECT <id>:<unit> <endpoint>`

Endpoint	Is
`console`	the host terminal. Exactly one unit may hold it.
`null`	nowhere. Writes vanish, reads never come.
`loopback`	itself — what you write comes back.
`scripted`	a caller in place of a human — what MCP and the tests type into.
`socket:PORT`	**listens** — this is telnet-in.
`socket:HOST:PORT`	**calls out**.
`serial:DEVICE`	a real serial port on this host.
`in:PATH`	a host file as a reader (paper tape). `?cps=N` paces it.
`out:PATH`	a host file as a punch — 8-bit clean, never truncating.
`terminal`	a window the simulator draws itself (SDL builds). `?emulation=vt100|adm3a|vt52|h19`, `?size=COLSxROWS`.
`printer:QUEUE`	a real print queue on this host.
`<endpoint>|FILE`	a tap: append `|FILE` to any endpoint to also log the line.
`<endpoint>|socket:PORT`	a live mirror: `telnet` in to watch and take over. `?ro` = watch-only.